

Technical Requirements Document (TRD)

Document 02

Status	Draft
Version	1.4 · 2026-10-02 02:38 · change log
Related	01 PRD , 04 App Flow , 05 Backend Schema

Contents

1	Architecture overview	1
2	Model strategy	1
3	Segment length policy	4
4	Corpus pipeline	4
5	Inference pipeline	5
6	Service design	6
7	Dependencies	8
8	Evaluation	8
9	Security and privacy	10
10	Testing strategy	10
	Change log	11

1 Architecture overview

Principles:

1. **Human text is authoritative.** Model output is stored as a *hypothesis* next to reviewer text and is never written over it.
2. **Immutability.** Source audio and transcripts are content-addressed (sha256) and never modified. Datasets are frozen snapshots, and model versions point to exactly one snapshot.
3. **One process model.** A single user on CPU needs no Redis or Celery. A worker process polls a jobs table in SQLite.
4. **Engines behind interfaces.** Aligner, VAD and ASR are swappable: a Protocol plus a registry, so the CTC comparison model or a larger Whisper plugs in without UI changes.

2 Model strategy

2.1 Why not “a proper English model + a proper Swahili model, blended”

This was the proposal in the requirements discussion. It does not work for Kenyan code-switching, for three reasons:

Local machine (CPU)

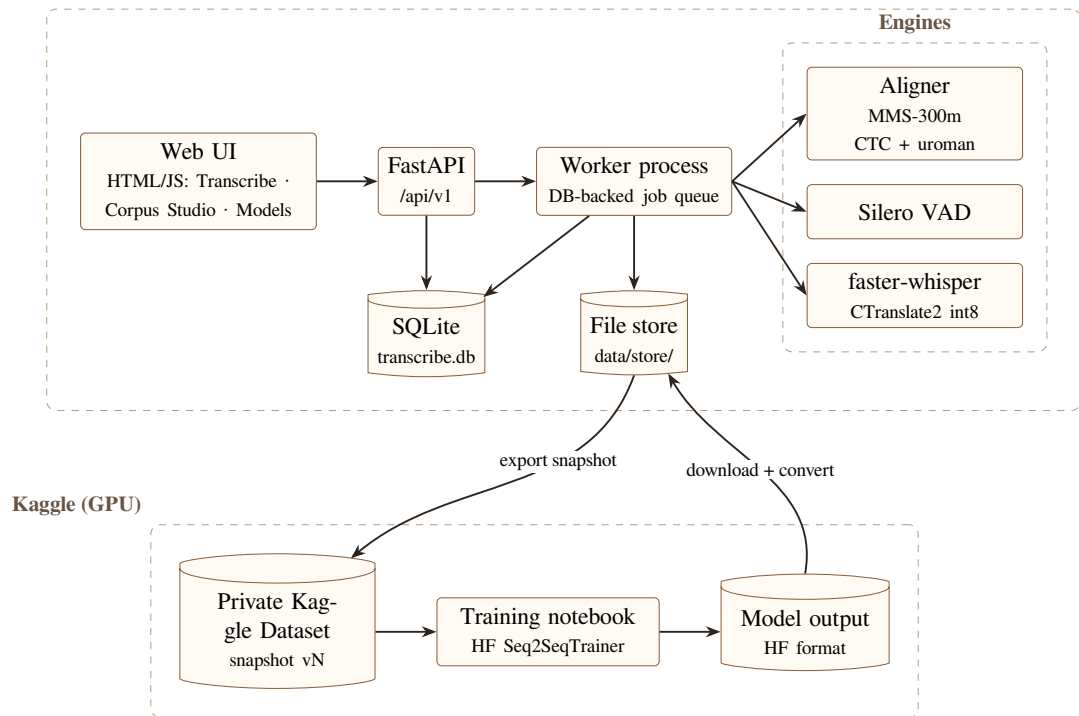


Figure 1: System architecture: local CPU services and Kaggle GPU training.

- **Routing needs per-word language ID before transcription.** A router (language ID → English or Swahili model) decides per segment. Switches happen mid-sentence and *mid-word* (*nimedownload*), below any practical language-ID granularity. Every routing error becomes a transcription error.
- **Monolingual models cannot output the other language.** A Swahili model spells English words in Swahili orthography (*daunlodi*), and an English model maps Swahili words onto English lookalikes. Merging hypotheses (ROVER-style) needs both hypotheses to contain the right word somewhere, and for intra-word switches neither does.
- **Cost and complexity.** It needs twice the compute on CPU, two fine-tuning tracks, and a merge heuristic with no training signal.

What we do instead: one multilingual pretrained model whose pretraining already covered both languages, fine-tuned on Kenyan code-switched speech. The blending happens in two places where it works:

- **The data mixture.** In-domain code-switched segments, plus public Kiswahili and public Kenyan-accented English, blended by sampling weights. This teaches the switch and keeps both monolingual abilities.
- **The tokenizer.** Whisper's byte-level BPE can emit any Latin string, so *nime + download* is just a token sequence. Nothing forces a single language per word.

2.2 Chosen models

Changed in v1.2: see the change log.

Role	Model	Why
v1 production	openai/whisper-small (244M), full fine-tune	Fits a Kaggle T4 in fp16. Readable output with casing and punctuation. Its 30 s window matches our segments. Mature CPU serving via faster-whisper int8. The fast iteration suits learning the process

Role	Model	Why
v2 candidate	openai/whisper-large-v3-turbo (809M, 4-layer decoder), LoRA or 8-bit optimiser	Much stronger encoder with an inference cost close to small/medium. Adopted only if it beats v1 on the frozen dev set by a meaningful margin
Comparison (learning track)	facebook/w2v-bert-2.0 + CTC head, optional KenLM trained on code-switched text	Contrasting paradigm: no hallucination, faithful verbatim output, character-level. Teaches CTC fine-tuning and LM fusion. Not production because it has no punctuation and weaker CPU tooling
Aligner (not trained)	MMS-300m forced-alignment CTC model (MahmoudAshraf/mms-300m-1130-forced-aligner) loaded with Hugging Face transformers; the Viterbi alignment is our own NumPy code (pilot/ctc_align.py). Latin EN/SW text needs no uroman: NFKD normalisation strips diacritics	Language-agnostic: it aligns any romanised text, so mixed EN/SW transcripts align without choosing a language. Owing the Viterbi removes the torchaudio deprecation risk and lets us add the two-pass anchoring of §4.3
VAD	Silero VAD	Robust on noise and music compared with the current energy threshold. Runs on CPU in ms per window
Legacy	Existing MFCC CNN-LSTM CTC/Seq2Seq	Kept under legacy/ as an educational baseline only. Not deployable (see §2.4)

Uncertainty on the aligner: torchaudio has been deprecating parts of its API. The aligner sits behind an interface and its dependency is pinned, so either backend can be used.

2.3 Whisper fine-tuning specifics

- **Language token:** fixed `<|sw|>` + `<|transcribe|>` + `<|notimestamps|>` for all training samples. Kenyan code-switching is predominantly Kiswahili-framed, and a fixed token removes the language-ID step at inference. **Experiment E3** compares sw vs en vs no language token on the dev set. This is a cheap and instructive ablation.
- **Non-speech examples:** about 5% of training samples are music/noise/silence windows with an empty target. This is the most direct fix for church and TV hallucination.
- **Data mixture (v1):** in-domain approved segments weight 1.0. Public Kiswahili (Common Voice sw, FLEURS sw_ke) weight about 0.3, to prevent forgetting and add speaker diversity. Kenyan-accented English (AfriSpeech-200 Kenyan subset) weight about 0.2. Licences must be checked before any commercial use (AfriSpeech is non-commercial).
- **Hyperparameters (starting point, T4 16 GB):** fp16, lr 1e-5, warmup 500, per-device batch 16 with gradient checkpointing, effective batch 32, 3–5k steps, evaluate every 500 steps on dev WER, keep the best checkpoint. SpecAugment is on (mask_time_prob=0.05).
- **Kaggle constraints:** a weekly GPU quota (~30 h) and a hard per-session limit, so we save checkpoints to /kaggle/working every 500 steps, the training script supports `resume_from_checkpoint`, and one session does one experiment.

2.4 Why the legacy model cannot be the product

- Hop 512 at 16 kHz gives 32 ms frames, and the three pooling layers make that 256 ms per CTC output frame. A 30 s segment has about 117 frames against about 400 target characters, and CTC requires frames $\geq$ target length. `CTCLoss(zero_infinity=True)` silently zeroes these losses.
- The tokenizer removes spaces, so output cannot be readable.
- It has no pretraining, and hundreds of hours of audio are needed before from-scratch ASR becomes competitive with a fine-tuned pretrained model.

If kept for learning, fix it with hop 160 (10 ms), 2× downsampling, a space token, and no SOS/EOS in CTC targets.

3 Segment length policy

(Answers “what chunk length?”)

Parameter	Value	Reason
Max segment	30.0 s	Whisper's encoder window is exactly 30 s; anything longer is truncated
Target	8–20 s	Enough context for the decoder; still fits CTC memory for the comparison model
Min segment	1.5 s	Shorter segments carry little context and add label noise
Split points	Aligned inter-word gaps ≥ 300 ms; prefer the largest gap; prefer turn boundaries	Never cut a word; natural prosodic units
Padding	100 ms on each side, clipped to the neighbouring word midpoint	Avoids clipped onsets without leaking neighbouring words
Speaker	A segment never spans a speaker turn change unless the turn is < 1.5 s	Clean targets; overlap is flagged

At inference, Silero VAD speech regions are merged into windows of ≤ 30 s. Windows are cut at the longest silence and overlap by 0 s (VAD-cut, so no stitching is needed).

4 Corpus pipeline

```

ingest → normalise audio → parse transcript → normalise text for alignment→
long-form alignment → word timestamps + scores → segmentation→
auto-flags + priority score → review → approve → dataset snapshot

```

4.1 Ingest

- ffmpeg decodes to a 16 kHz mono 16-bit WAV. The original file is kept byte-identical, stored at data/store/originals/<sha256>.<ext>.
- The transcript is parsed by the existing docx_parser / pdf_parser. The speaker regex and language hints (italic → sw, a convention that is verified per source) are reused.

4.2 Text normalisation for alignment (a separate text, never shown as the label)

Changed in v1.2: see the change log.

- Remove transcriber markers and record them on the neighbouring word. Real transcripts use several forms, all recognised: [Unclear 13:10-13:15], [No speech - activity, 17:22-17:24], [[Unclear 1:16:07]] and (Unclear: 1:16:07-1:16:08)]. Their **timestamps are kept** and used as alignment boundaries and as an accuracy check (§4.3).
- Participant references such as P7. stand in for a spoken name: they are not aligned, and the segment is flagged redacted_marker (excluded from training).
- False starts and cut-offs written with - -, an em dash or an ellipsis (tuta - - tutaanza) are split into separate words. Punctuation-only tokens stay attached to the neighbouring word.
- Lowercase and strip diacritics (NFKD); keep only letters in the aligner's vocabulary.
- **Numbers (current pilot):** digits are not aligned; their words get times interpolated from neighbours,

and the segment is flagged `has_number`. **Planned (Phase 1):** verbalisation per [STYLE_GUIDE §4](#), generating an English and a Kiswahili candidate and keeping the one the aligner scores higher; near ties flag `number_check`. English via `num2words`, Kiswahili via a rule-based corpus/`numbers_sw.py`.

- Optional display ITN (words → digits) runs on Transcribe output only (P1), never on training text.
- Each word keeps its original spelling, which becomes the segment label; the aligner only sees the normalised letters.

4.3 Long-form alignment

Changed in v1.2: see the [change log](#).

Validated on NRCCW_KSM09 (2 h 05 min, 1 237 turns, 12 305 words); see the [change log](#) for the measurements.

- **Emissions.** CTC log-probabilities for the whole recording in 30 s windows with 2 s context, stitched and cached (`emissions.npy`, about 23 MB for 2 h). This is the only slow step: about $0.2\times$ real time on a 4-core CPU, $0.8\times$ on a slower laptop, a few minutes on a Kaggle GPU.
- **Confidence** is *relative*: for each letter, how close the forced path is to the aligner’s best guess, measured only on frames where the path emits a letter or the model hears one. Silence agreement proves nothing and is ignored. Spoken text scores near 1 even in noisy far-field audio; text that was never spoken scores low (FLEURS mock: spoken 0.70–1.00, unspoken 0.41). The earlier absolute-posterior measure rejected most real turns on noisy audio.
- **Pass 1, anchors.** Each turn of at least 12 letters is searched in a window after the last trusted turn. It becomes an anchor only if its confidence clears a length-dependent bar (0.8 for short turns, easing to 0.5 for long ones), because short text can match by chance. Shorter turns (“Mm-hmm”) are never anchors.
- **Pass 2, fill.** Everything between two consecutive anchors is aligned as one block, bounded on both sides, so it cannot drift and short turns are placed in context. The block is split at timestamped markers: words before a marker must end near its start and words after it must start near its end (± 3 s tolerance). Turns with no room between their anchors (usually backchannels spoken over the other person) stay *unfilled*.
- **Safety.** Timings that run backwards are dropped. Words whose confidence is below 0.5 make their segment `low_alignment`, which keeps it out of training. On NRCCW_KSM09 every word placed more than 10 s from a transcriber timestamp fell in such a segment.
- **Accuracy check.** The report compares aligned times with the transcriber’s timestamps (median error, share within 3 s and 10 s). Tuning used half of the markers as boundaries and scored on the other half.
- Output per word: start, end, score (relative confidence, 0–1).

4.4 Priority score for the review queue

```
priority = 0.5 · (1 - alignment_conf) + 0.2 · has_unclear_marker + 0.1 · overlap_marker
          + 0.2 · model_disagreement (WER(aligned_text, model_hypothesis), once a model exists)
```

The review queue sorts by priority descending. Segments with high confidence and low disagreement can be **batch-approved after spot-checking** a random 10%, controlled by a setting and off by default.

4.5 Recordings without transcripts

Silero VAD splits them into ≤ 30 s chunks, the current production model pre-fills the text (stored as a hypothesis, with the reviewer text starting from a copy), and they go into the same review UI.

5 Inference pipeline

1. Decode with `ffmpeg` → 16 kHz mono float32.
2. Silero VAD (threshold 0.5, min speech 250 ms, min silence 400 ms) gives speech regions.
3. Merge regions into windows of ≤ 30 s.

4. Run faster-whisper on each window: `language="sw", task="transcribe", beam_size=5, condition_on_previous_text=False, word_timestamps=True, vad_filter=False` (VAD has already been applied).
5. Guards: drop segments with `no_speech_prob > 0.6` and `avg_logprob < -1.0`. Drop segments with compression ratio `> 2.4` (repetition loops). Collapse a 3-gram repeated more than 3 times.
6. Post-processing: timestamps are offset back to absolute time, and segments are persisted as `job_segments`.
7. Export renderers: TXT, SRT, VTT, JSON, DOCX.

Batching: windows run sequentially, with `cpu_threads = physical cores` and `compute_type="int8"`.
 Progress = processed audio seconds / total speech seconds.

6 Service design

6.1 Processes

Changed in v1.4: [see the change log](#).

Process	Command	Responsibility
api	<code>python -m transcribe.api</code>	FastAPI serving the HTML/JS pages at / and /api. Handles CRUD and enqueues jobs
worker	<code>python -m transcribe.worker</code>	Polls jobs (<code>status='queued'</code>), runs alignment, transcription, evaluation and export, and heartbeats

Both processes share SQLite in WAL mode. A job is claimed with `UPDATE jobs SET status='running', worker_id=? WHERE id=? AND status='queued'`. A stale heartbeat (`> 2 min`) puts the job back in the queue.

6.2 REST API (/api/v1)

Method	Path	Purpose
POST	<code>/recordings</code>	Multipart upload of audio (+ transcript), domain, consent_scope
GET	<code>/recordings, /recordings/{id}</code>	List and detail, with review progress
POST	<code>/recordings/{id}/align</code>	Enqueue alignment + segmentation
GET	<code>/review/queue?recording_id=&limit=</code>	Next segments by priority
GET	<code>/segments/{id}</code>	Segment + current revision + hypotheses + audio URL
POST	<code>/segments/{id}/revisions</code>	Create a new revision (text, bounds, flags, spans, status). Rejected with 409 on version conflict
POST	<code>/segments/{id}/hypothesis</code>	Run the model on the segment (stored as a hypothesis only)
GET	<code>/segments/{id}/audio</code>	WAV slice (rendered on demand from the recording, cached)
POST	<code>/datasets</code>	Build a snapshot from a filter spec + split spec
POST	<code>/datasets/{id}/export</code>	Write the HF-format package for Kaggle
POST	<code>/models</code>	Register a model (upload or path), enqueue CT2 conversion
POST	<code>/models/{id}/promote</code>	Set the production model
POST	<code>/evaluations</code>	Enqueue evaluation of a model on a dataset split
GET	<code>/evaluations/{id}</code>	Metrics + per-segment results
POST	<code>/transcriptions</code>	Upload a file for transcription (enqueue a job)
GET	<code>/transcriptions/{id}</code>	Status, progress, segments

Method	Path	Purpose
PATCH	/transcriptions/{id}/segments/{seg_id}	Save an edit
GET	/transcriptions/{id}/export?format=srt	Download
POST	/transcriptions/{id}/promote	Copy edited segments into the corpus (consent-checked)

Optimistic concurrency: every revision POST carries `base_revision_id`. A mismatch returns 409, so a second browser tab cannot overwrite silently.

6.3 Transcription app and live adaptation

Changed in v1.4: see the change log.

python -m transcribe.app serves three pages: *Load audio*, *Transcription in progress* and the chunk editor (UI/UX §4.1–4.2). Sessions live in `data/app/sessions/<id>/` (original upload, 16 kHz WAV, `session.json`); `session.json` has the shape of the pilot’s review manifest, so `migrate_json` and `review_stats` read it unchanged. Two worker threads share one Whisper instance per model: one runs whole transcriptions, the other runs look-ahead re-transcription.

Pipeline. Any audio or video is decoded with PyAV to 16 kHz mono; Silero VAD finds speech; regions are merged into chunks of up to 30 s, closing a chunk at a pause of at least 0.6 s once it is 15 s long; each chunk is transcribed with the chosen language, a token cap proportional to its duration (about 12 tokens/s) and three retry temperatures, which stops repetition loops from running for minutes; speakers are labelled by `pyannote` when installed and `HF_TOKEN` is set.

Learning while the reviewer works. Weights are not changed live. A fine-tuning step on ten short clips overfits them and can make the next chunks worse, with no held-out check to notice; on a CPU one step takes about a minute. Instead, after each save:

1. **Correction rules.** One-word substitutions the reviewer made at least twice (*sababa* → *sababu*), and kept uncorrected less often than corrected, are applied to every chunk not yet edited.
2. **Prompt and hotwords.** The last 220 characters of corrected text, and words the reviewer typed that the model never produced (names, hybrid spellings), are passed to Whisper as `initial_prompt` and `hotwords`.
3. **Look-ahead.** The next three unedited chunks are re-transcribed in the background with that context; a chunk the reviewer has edited is never overwritten, and an unedited chunk’s text is replaced only on the server.

Weights change in batches: every corrected chunk is training data for the Kaggle fine-tuning runs of Phase 4.

6.4 Package layout (target)

Changed in v1.4: see the change log.

```
src/transcribe/
  api/           # FastAPI app, routers, schemas (pydantic)
  worker/       # job loop, handlers
  db/           # SQLAlchemy models, session, Alembic migrations
  corpus/
    ingest.py   parsers/ (docx_parser.py, pdf_parser.py – moved from preprocessing)
    textnorm.py align.py segment.py priority.py
  inference/
    vad.py      engine.py (Protocol + faster-whisper impl) longform.py guards.py export/
  datasets/     # snapshot build, grouped split, HF export
```

```

finetune/      # whisper_train.py, w2vbert_ctc_train.py (run inside Kaggle)
evaluation/    # normalizer.py, metrics.py (WER/CER/switch-point/hallucination), report.py
app/          # transcription app (built): server, pipeline, adapt, store, static/ (HTML,
    ↪ CSS, JS)
ui/           # Corpus Studio and Models pages (later; same plain HTML/JS approach)
legacy/       # ctc_model.py, seq2seq.py, trainer.py, data/ (unchanged behaviour)
notebooks/kaggle/ # thin notebooks calling transcribe.finetune.*
tests/

```

Current layout (after Phase 0). The package installs as `transcribe` (`python -m pip install -e .`); tools run as `python -m transcribe....`, without the old `src.` prefix or `sys.path` hacks. What exists today: `pilot/` (alignment, segmentation, metrics, Whisper runner, reports, Kaggle batch), `app/` (the transcription app), `db/` (models, migrations, invariants), `preprocessing/` (DOCX/PDF parsers, energy VAD, chunk export, manifests), `script/` (corpus CLIs) and `legacy/` (the from-scratch baseline: `models/`, `training/`, `data/`, `evaluation/`, `config.py`, and `train`, `evaluate`, `infer`). The pilot modules move into `corpus/` and `evaluation/` in Phases 1 and 3.

The package imports as `transcribe` (installed via `pip install -e .`). The `sys.path / src.` import hack is removed.

7 Dependencies

Changed in v1.4: see the change log.

Group	Packages
core	fastapi, uvicorn, sqlalchemy≥2, alembic, pydantic≥2, python-multipart, soundfile, numpy, ffmpeg (system)
corpus	python-docx, pypdf, transformers<5 (aligner model), torch; librosa + soundfile for audio
pilot (now)	requirements-pilot.txt: the annotation requirements plus transformers, faster-whisper, jiwer
inference	faster-whisper, silero-vad (onnx), python-docx (export)
ui	fastapi, uvicorn, python-multipart
eval	jiwer
finetune (Kaggle only)	transformers, datasets, accelerate, evaluate, peft (v2), bitsandbytes (v2)
legacy	librosa, scikit-learn, editdistance, nltk

Changed in v1.3: see the change log.

Requirements are grouped in `requirements/{core,pilot,ui,legacy,dev}.txt`, mirrored as extras in `pyproject.toml` (`pip install -e ".[pilot,dev]"`). The root `requirements.txt`, `requirements-annotation.txt` and `requirements-pilot.txt` remain for existing commands; each ends with `-e .` so it also installs the package. The *corpus*, *inference*, *eval* and *finetune* groups in the table are added as those phases start. Versions are pinned with upper bounds.

8 Evaluation

8.1 Splits (answers “how should we split?”)

Changed in v1.1: see the change log.

- **Group by recording, never by chunk.** Chunks from one recording share speaker, microphone, room and topic, so a random chunk split leaks all of them and inflates results.

- If a speaker appears in several recordings (the same interviewer), group by speaker cluster: all recordings of that speaker go to the same split, or that speaker is held out explicitly.
- **Size dev/test in hours and recordings, not percentages.** Evaluation sets only need to be big enough for a stable WER, and variance comes from speakers and recordings more than from word count. Long-run target: **test $\approx$ 3 h from $\geq$ 15 recordings, dev $\approx$ 2–3 h from $\geq$ 10 recordings.** Everything else is train; as audio keeps arriving, train grows and dev/test stay fixed.
- **Current corpus (v1): 11 recordings, 8 h 56 min.** test_v1 = 2 recordings ($\approx$ 1.5 h, 17%), dev_v1 = 1 recording ($\approx$ 50 min, 10%), train = 8 recordings ($\approx$ 6.5 h, 73%), including the 2 h 05 min recording. The test set is larger than dev because it is the number every model version is judged on; dev only has to rank checkpoints. With so few recordings a single test pair is noisy, so final v1 numbers are also reported as leave-two-recordings-out cross-validation over 3 folds (3 Kaggle runs), and the frozen split is kept for version-to-version comparison.
- **Shared speakers:** recordings are mostly of different people, but some people may appear in more than one recording. Recordings that share any person (interviewer or respondent) form one *speaker group*, and a whole group always goes to the same split. Groups come from the file-naming convention, cross-checked by a speaker-embedding similarity pass (ECAPA-TDNN embeddings per recording, cosine similarity above a tuned threshold flags a probable shared speaker for manual confirmation). If an interviewer still ends up in several splits, reports add a *respondent-only* WER.
- **Test is frozen forever** (for comparability across model versions), stratified so it contains clean and noisy, low and high code-switch density, and each domain available. Dev is used for checkpoint selection and ablations.
- **Audio still arriving:** fix test_v1 and dev_v1 from the recordings available at the end of Phase 2, and never change them. New recordings go to train by default. A new domain or a clearly new condition (TV, church, a new region) first contributes $\approx$ 1 h to an external_*<domain>* test set.
- New recordings go to train by default. New *domains* (TV, church) first get a held-out evaluation set of $\geq$ 1 h before any of their data goes into train.
- Also keep FLEURS sw_ke test as a forgetting check. Test-set segments are never used for anything except final reports.

8.2 Normaliser (versioned, normalizer_version stored with each evaluation)

Lowercase. Strip punctuation except intra-word apostrophes. Remove hyphens (so *nime-download* $\equiv$ *nime-download*). Collapse whitespace. Remove bracketed annotations. Map a small table of spelling variants (e.g. *ok/okay*) from the style guide. Verbalise any remaining digits with the same rules as the corpus pipeline, so digit and word outputs score the same.

8.3 Metrics

Metric	Definition
WER / CER	jiwer on normalised text, micro-averaged (total errors / total reference words)
Per-language WER	Using reference language spans: errors attributed to en, sw and mixed reference words via the jiwer alignment
Switch-point error rate	Over reference words adjacent to a language change (plus all mixed words): the fraction misrecognised. This directly measures the code-switching capability
Hallucination rate	Words emitted per minute on reference non-speech segments (the non-speech test subset)
RTF	Wall time / audio duration on the reference CPU

Reports are broken down by domain and by flag (clean / noisy / overlap).

9 Security and privacy

- The app binds to 127.0.0.1 by default. Exposing it on a network requires setting `TRANSCRIBE_AUTH_TOKEN` (a bearer token middleware) in v1.
- `consent_scope` $\in$ {training, eval_only, transcription_only} on every recording is enforced in the dataset builder (training split only from training), in the Kaggle export, and in “promote to corpus”.
- Kaggle datasets are created private. The export package contains only audio slices and text, never names from the transcript header or file paths.
- No demographic or ethnicity fields are stored. Speaker labels are opaque (S1, interviewer).
- Uploaded transcription files are deleted after N days (configurable) unless promoted.

10 Testing strategy

Changed in v1.2: see the change log.

Level	What
Unit	Text normaliser, segmenter on synthetic word timings, priority, grouped split (asserts no recording in two splits), metric functions (hand-computed cases incl. switch points), guards
Integration	A 60 s synthetic fixture (TTS or public-domain clip + transcript) goes through align $\rightarrow$ segment $\rightarrow$ dataset export, all on CPU in < 2 min
Pilot (now)	24 tests in <code>tests/pilot</code> : text preparation on real transcript quirks, the Viterbi, segmentation, metrics, file pairing, a 200-turn alignment regression with unspoken turns, and an end-to-end run with a fake aligner and fake Whisper
Smoke test	<code>pilot.make_mock</code> : a 4-minute interview stitched from public FLEURS Kiswahili and English clips, with one written-but-unspoken turn that must be the only one flagged
API	FastAPI TestClient: revision conflict (409), consent enforcement, job lifecycle
Model	The Kaggle notebook prints dev WER. <code>evaluate</code> reproduces the same WER locally within ± 0.5 absolute
CI	GitHub Actions: <code>ruff</code> + <code>pytest</code> (unit + integration with a tiny aligner mock) on Ubuntu, Python 3.10

Change log

Times are East Africa Time (EAT, UTC+3). Each entry names the affected section, quotes what it said before and summarises what it says now, and links to the previous text: on GitHub at the exact commit and lines, and as an archived PDF of the previous version.

Version	Date (EAT)	Commit	Summary
1.0	2026-09-24 00:36	053e09d	First LaTeX version (converted from the Markdown draft of 2026-09-23).
1.1	2026-09-24 01:53	695b407, 8a3be55	Splits sized for the real corpus (11 recordings, 8 h 56 min); split by speaker group.
1.2	2026-09-24 22:26	c28ea1d	Pilot results: aligner implementation, text preparation, two-pass long-form alignment, dependencies, tests.
1.3	2026-10-02 00:13	this commit	Phase 0: current package layout and requirement groups.
1.4	2026-10-02 02:38	this commit	Transcription app: web UI instead of Gradio, pipeline, live adaptation; layout and dependencies.

v1.4 • §6.4 Transcription app and live adaptation (new)

Previous text: none (new subsection) · [v1.3 PDF](#)

Before. Nothing on the app or on learning from corrections.

Now. How the app is built (pipeline, sessions, workers) and why corrections adapt prompts and rules, not weights, until the batch fine-tuning of Phase 4.

v1.4 • §6.1 Processes; §6.3 Package layout; §7 Dependencies

Previous text: [v1.3 source](#), line 254 · [v1.3 PDF](#)

Before. “FastAPI + Gradio mounted at /”; layout with ui/ # Gradio Blocks; dependency row ui: gradio $\geq$ 4; current layout lists review/ (Gradio review app).

Now. FastAPI serves HTML/JS pages and /api; layout adds app/ and the later ui/ in plain HTML/JS; dependency row fastapi, uvicorn, python-multipart; review/ removed (replaced by app/). Architecture figure relabelled “Web UI”.

v1.3 • §6.3 Package layout

Previous text: [v1.2 source](#), lines 294–313 · [v1.2 PDF](#)

Before. Only the target layout (api/, worker/, db/, corpus/, ..., legacy/).

Now. Keeps the target and adds the current layout after Phase 0: installable transcribe package run as python -m transcribe...; pilot/, preprocessing/, review/, script/, and the from-scratch baseline moved to legacy/.

v1.3 • §7 Dependencies

Previous text: [v1.2 source](#), line 338 · [v1.2 PDF](#)

Before. “Requirements are split into requirements/{core,corpus,inference,ui,finetune,legacy}.txt.”

Now. Actual groups: core, pilot, ui, legacy, dev, mirrored as pyproject.toml extras; root requirement files kept and install the package; remaining groups added per phase.

v1.2 • §2.2 Chosen models: aligner row

Previous text: [v1.1 source](#), line 74 · [v1.1 PDF](#)

Before. “MMS-300m forced-alignment model + uroman romaniser (via ctc-forced-aligner, or torchaudio MMS_FA pinned).”

Now. The same MMS-300m aligner, loaded with transformers; our own NumPy Viterbi; no uroman (NFKD normalisation). Removes the torchaudio deprecation risk and enables two-pass anchoring.

v1.2 · §4.2 Text normalisation for alignment

Previous text: [v1.1 source, lines 156–171](#) · [v1.1 PDF](#)

Before. Remove bracketed annotations; romanise with uroman; verbalise numbers with English/Kiswahili candidates chosen by the aligner; keep a word-index map.

Now. Recognises the marker forms found in the real transcript and **keeps their timestamps**; treats P7-style participant references as redactions; splits false starts (tuta--tutaanza); records that number verbalisation is still planned and the pilot interpolates digits and flags `has_number`.

v1.2 · §4.3 Long-form alignment

Previous text: [v1.1 source, lines 173–186](#) · [v1.1 PDF](#)

Before. “Alignment is turn-anchored: each transcript turn is aligned inside a search window around its expected position (moving anchor)... score (mean frame log-posterior...) A turn whose mean confidence is below θ (initially 0.5) is flagged `low_alignment`.”

Now. Relative confidence on informative frames; pass 1 trusts only confident anchor turns (length-dependent bar); pass 2 aligns each block between anchors, split at transcript timestamps; backwards timings dropped. On NR-CCW_KSM09: words with timings 6 643 → 11 590 of 12 305; segment hours 1.04 → 1.70; clean hours 0.82 → 0.91; no word over 30 s from a transcriber timestamp (was 2 of 14 placed); held-out median error 1.3 s.

v1.2 · §7 Dependencies

Previous text: [v1.1 source, lines 312–318](#) · [v1.1 PDF](#)

Before. Corpus group: “python-docx, pypdf, ctc-forced-aligner (or torchaudio pinned), uroman, torch (CPU)”.

Now. Corpus group uses transformers; new *pilot* row for requirements-pilot.txt (transformers, faster-whisper, jiwer).

v1.2 · §10 Testing strategy

Previous text: [v1.1 source, lines 402–407](#) · [v1.1 PDF](#)

Before. Unit, integration, API, model and CI rows only (all planned).

Now. Adds what exists now: the 24 pilot tests and the FLEURS mock-interview smoke test.

v1.1 · §8.1 Splits

Previous text: [v1.0 source, lines 334–341](#) · [v1.0 PDF](#)

Before. “Target: test $\approx$ 3 h from $\geq$ 15 recordings, dev $\approx$ 2–3 h from $\geq$ 10 recordings. Everything else is train. With 40 h that is about 85/7/8.”

Now. Keeps that as the long-run target and adds the v1 split for the real corpus (11 recordings, 8 h 56 min): test 2 recordings ($\approx$ 1.5 h), dev 1 ($\approx$ 50 min), train 8 ($\approx$ 6.5 h), with 3-fold leave-two-recordings-out cross-validation. Recordings sharing a person form a *speaker group* that never spans splits (naming convention plus an ECAPA speaker-embedding check).